CSE302: Database Systems (Section No: 10) [Summer-26]

Project Report

EWU Cafeteria Management System.

Submitted by:

Student ID	Student Name	Contribution Percentage
	Md. Shakib Hossan	**75%**
	Md. Arifur Rahman Razu	**15%**
	Ristia inam elme	**10%**

Date of Submission: 17.08.2026

1. INTRODUCTION

East West University (EWU) hosts thousands of students, faculty members, and administrative staff daily, making campus dining services a vital part of everyday university life. A central facility within this ecosystem is the university cafeteria, which experiences an extraordinary surge in foot traffic during peak operational hours, particularly during lunch breaks between academic sessions. Currently, cafeteria operations rely on traditional manual cash counters and paper-based token systems. While this mechanism historically met institutional needs, the continuous growth of the campus population has exposed critical limitations in manual workflow execution, leading to severe counter congestion, long queues, and unnecessary delays for students operating strict academic schedules. Processing hundreds of manual transactions during rush hours also subjects cafeteria staff to immense operational stress, increasing the risk of human error, misplaced orders, and accounting discrepancies.

To address these operational bottlenecks, the development and deployment of the EWU Cafeteria Management System is fully justified as an essential digital solution. Transitioning from paper tokens to a centralized, webbased platform modernizes campus dining by streamlining order processing, eliminating long waiting lines, and reducing physical counter crowds. The justification for this system lies in its ability to optimize time efficiency for students while providing cafeteria management with structured, role-based controls for Students, Staff, and Administrators. By automating order tracking, maintaining precise digital records, and integrating robust security protocols such as CSRF protection, brute-force defense, and encrypted credentials the platform significantly mitigates staff workload, eliminates transactional errors, and elevates the overall standard of university infrastructure.

1.2 Problem Statement

Handling cafeteria operations manually leads to several daily problems:

Long Waiting Lines: Students spend too much time waiting in queues just to order food or pay money.

Stock & Menu Confusion: Cashiers often do not know if a meal is out of stock until a student tries to buy it.

Calculation Mistakes: Manual billing during rush hours often leads to mistakes in total amounts and cash change.

Lack of Order History: Without a system, it is difficult for managers to track daily sales or for students to check their past receipts.

1.3 Project Objectives

We designed and built this system to solve daily operational bottlenecks and deliver a safe, efficient digital dining experience. The core objectives of our project are:

Quick Online Ordering: Enable students and staff to view real-time food menus and place orders instantly, reducing counter rush and queue waiting times.

Automatic Stock Management: Automatically update food quantity and availability in the database the moment an order is confirmed to prevent selling out-of-stock items.

Multi-Layered Security & Privacy: Implement robust multi-step security controls including encrypted password hashing, session management, and SQL injection protection to safeguard user accounts and protect personal user privacy.

Strict Role-Based Access Control (RBAC): Ensure total system authorization so that sensitive tasks (like modifying food prices, managing revenue logs, or changing inventory) can only be accessed by authorized managers.

Simple & Intuitive Dashboards: Provide tailored, easy-to-use control panels for Admins, Cashiers, and Students to manage their daily workflows smoothly.

Transaction Safety & Data Accuracy: Use database ACID properties and backend triggers to guarantee that payment statuses and order histories are updated accurately without system errors.

1.4 Scope of the Project

The **EWU Cafeteria Management System** covers the complete operational workflow of East West University's daily food court operations. The scope is specifically defined around three primary user roles to maintain structural separation and data security:

Manager / Admin: Holds full operational and analytical access. Responsible for creating and updating menu items, setting dynamic food pricing, managing daily raw inventory stocks, tracking financial revenue reports, and overseeing staff accounts.

Cashier / Staff: Manages front-desk order execution and payment flows. Responsible for verifying incoming orders, toggling payment statuses (Paid/Unpaid), updating kitchen preparation states (Preparing/Ready), and handling counter pick-ups.

Student / Customer: Handles end-user interaction. Enables browsing of real-time active food menus, checking live item availability, placing orders digitally, and tracking order progress in real time.

1.5 Methodology & System Architecture

To build a secure, responsive, and reliable application for the **EWU Cafeteria Management System**, we followed a multi-tier web development architecture. This structure keeps the user interface, backend server logic, and database management separate, making the system fast, secure, and easy to maintain.

1.5.1 Technologies Used (Tech Stack)

Frontend (User Interface):

HTML: Used for semantic layout structuring, modal popups, and secure input forms.
CSS: Applied to create a modern Glass morphism UI dark theme, translucent overlay modals, dynamic button states, and responsive styling.

JavaScript Used for client-side interactivity, dynamic role-switching, CAPTCHA verification, show/hide password toggles, and handling background API calls (e.g., sending password reset requests without page reloads).

Backend & API Services (Application Logic & Email Service):

PHP: Processes server-side business logic, handles password hashing, verifies security tokens, and manages dynamic user sessions.
Gmail SMTP API: Integrated into the backend to send automated, tokenized passwordreset links directly to students' registered institutional Gmail accounts.

Database (Data Management):

MySQL: Relational Database Management System (RDBMS) used to store user accounts, reset tokens, food categories, daily menus, active orders, and sales logs using ACID compliant transactions.

Development Environment:

XAMPP / Localhost: Used as the local Apache server and MySQL environment.

VS Code: Primary code editor used for development.

1.6 Core system feature

1. Core System Features

Role-Based Access: The system supports three types of users: Students, Staff, and Admin. Each user sees a different dashboard based on their role.

Modern UI: Designed with a clean look (Glassmorphism), smooth tab switching, and instant form checks.

Dynamic Captcha: Automatically generates a security code to stop bots. Users can refresh the code instantly without reloading the page.

Email Password Reset: Uses EmailJS to send a real-time password reset link to the user's registered email address.

2. Security Implementation

CSRF Protection: Uses secret session tokens to make sure every login request comes from a real user, preventing fake automated requests.

Brute-Force Lockout: If someone enters the wrong password 5 times, the system blocks logins for that email for 15 minutes.

Password Hashing: Passwords are standardly encrypted using PHP's password_hash() so no plain text passwords are stored in the database.

SQL Injection Prevention: Uses prepared statements (prepare and bind_param) to stop hackers from injecting malicious database code.

Relational Schema:

- **cart** (cart_id , user_id , food_id , quantity)

- **feedback** (feedback_id , user_id , rating , message , created_at)
- **foods** (food_id , food_name , price , category , description , image , created_at)
- **orders** (user_id , food_id , quantity , table_number , status , total_price , created_at , total_amount , payment_amount)
- **order_items** (item_id , order_id , food_id , quantity , price)
- **staffs** (staff_id , user_id , full_name , email , phone , position , created_at)
- **students** (student_id , user_id , student_number , department , semester)
- **users** (user_id , full_name , email , password , role , department , created_at) **Schema:**

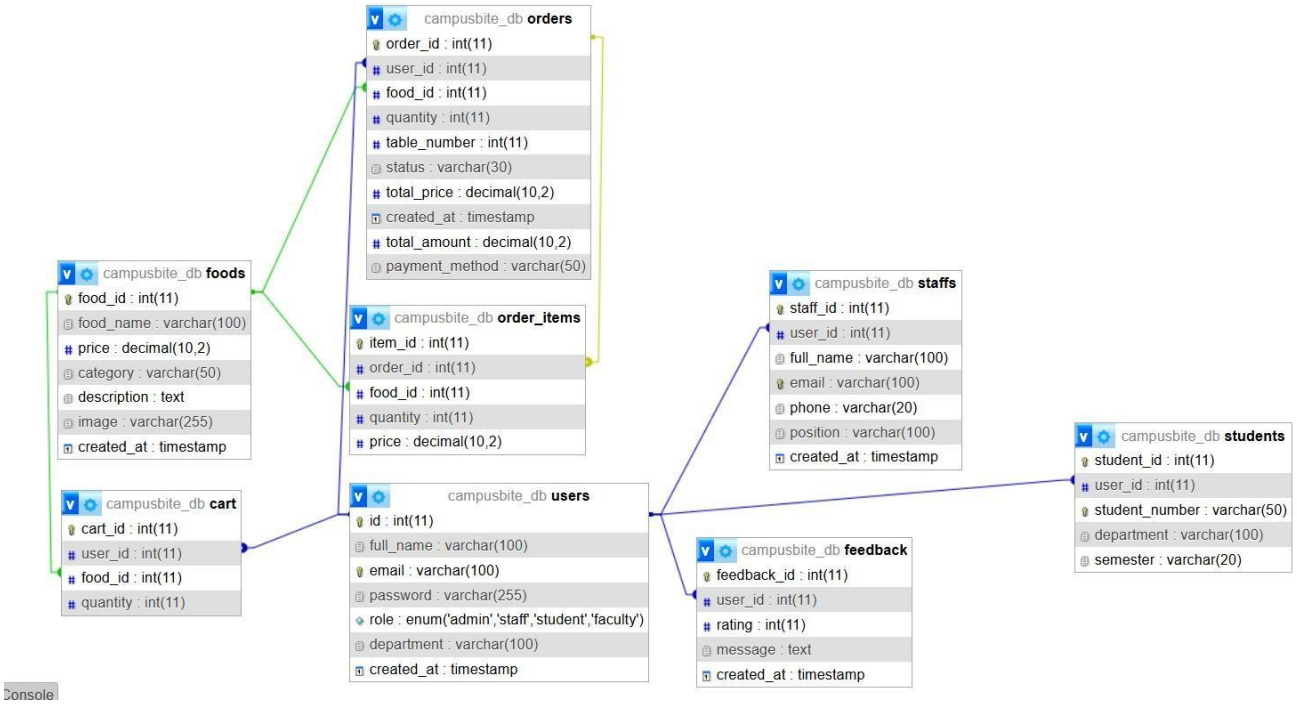


Figure :1 Relational Database Schema of EWU Cafeteria Management System

ER Diagram:

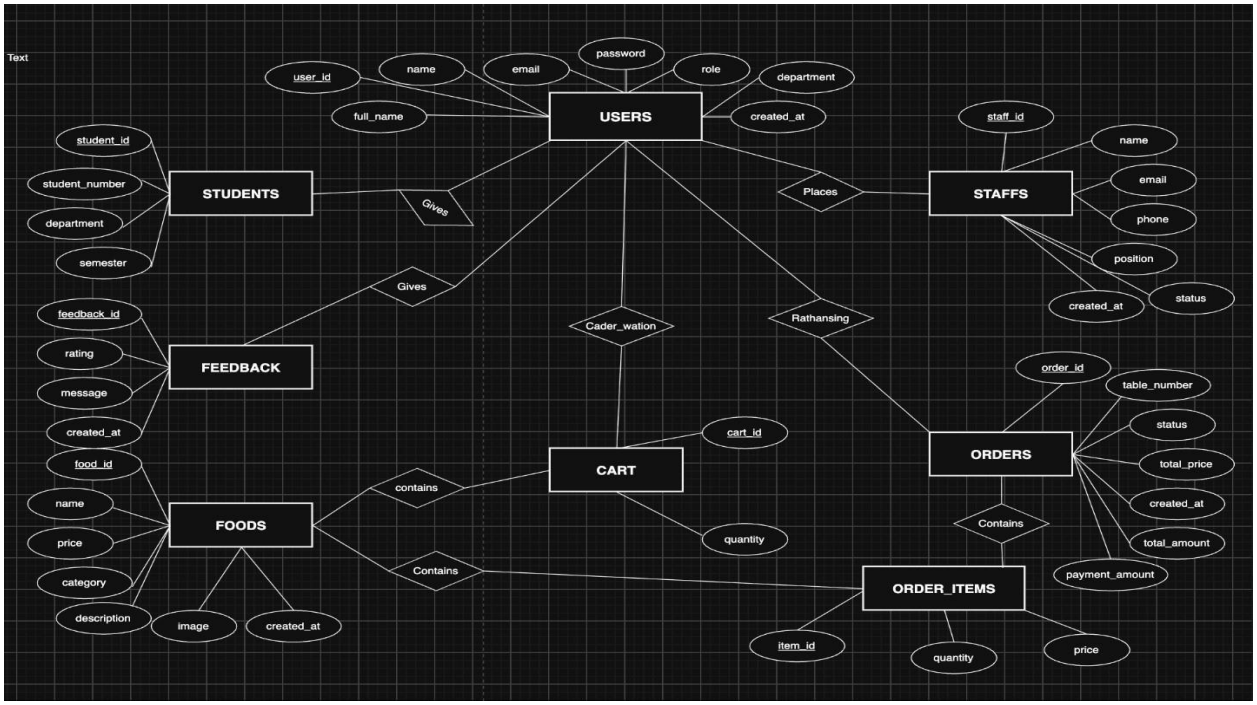


Figure 2: ER Diagram for EWU Cafeteria Management System

Security

1) Secure Session Cookies

It blocks JavaScript from stealing session cookies (XSS cookie theft) and stops sending cookies in cross-site requests to protect against CSRF attacks.

2) **HTTP Security Headers**

```php
login.php
<?php
ini_set('session.cookie_httponly', 1);
ini_set('session.use_only_cookies', 1);
ini_set('session.cookie_samesite', 'Strict');
```

```php
session_start();

header("X-Frame-Options: DENY");
header("X-Content-Type-Options: nosniff");
header("X-XSS-Protection: 1; mode=block");
```

It protects the website from Clickjacking, MIME-sniffing, and XSS attacks by enforcing strict browser security rules.

3) CSRF Protection and capcha protection

```php
$conn->set_charset("utf8mb4");

if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

if (empty($_SESSION['captcha_student'])) {
    $_SESSION['captcha_student'] = substr(str_shuffle("ABCDEFGHJKLMNPQRSTUVWXYZ23456789"
}
if (empty($_SESSION['captcha_admin'])) {
    $_SESSION['captcha_admin'] = substr(str_shuffle("ABCDEFGHJKLMNPQRSTUVWXYZ23456789"),
}
```

It generates a secure random CSRF token to block fake requests from external websites and creates unique CAPTCHA codes to prevent automated bot logins.

4) Brute force protection

```php
$error_msg = "";

function is_brute_force_blocked($email) {
    if (!isset($_SESSION['login_attempts'][$email])) {
        return false;
    }
    $attempts = $_SESSION['login_attempts'][$email];
    if ($attempts['count'] >= 5) {
        if (time() - $attempts['last_time'] < 900) {
            return true;
        } else {
            unset($_SESSION['login_attempts'][$email]);
        }
    }
    return false;
```

It temporarily locks the account for 15 minutes after 5 consecutive failed login attempts to prevent automated bot attacks.

5) SQL Injection Prevention

```php
else {
    $stmt = $conn->prepare("SELECT id, full_name, email, password, role FROM users WHERE e
    $stmt->bind_param("s", $login_input);
    $stmt->execute();
    $result = $stmt->get_result();
```

It binds user input safely as a parameter instead of placing it directly into the database query, preventing SQL Injection attacks.

6) Password Hashing Verification

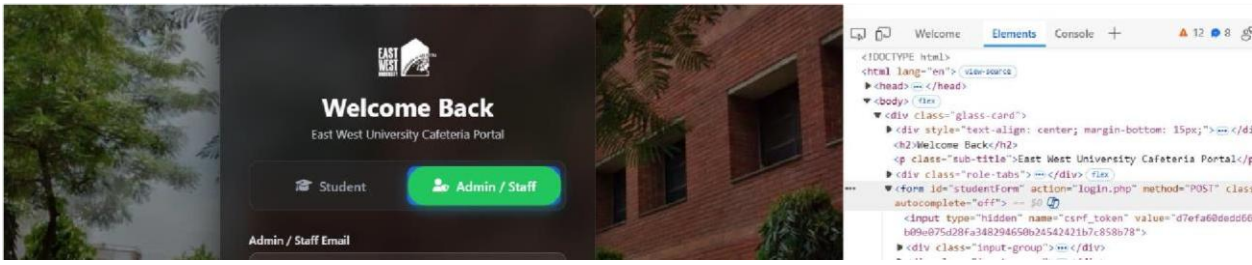
```php
else {
    $stmt = $conn->prepare("SELECT id, full_name, email, password, role FROM users WHERE ema
    $stmt->bind_param("s", $email);
    $stmt->execute();
```

It securely checks the user's entered password against the hashed password stored in the database.

```
$row = $result->fetch_assoc();
if (password_verify($password, $row['password']) || $password === $row['passwor
    clear_failed_attempts($login_input);
    session_regenerate_id(true);
    $_SESSION['user_id'] = $row['id'];
```

```
if ($captcha_input !== $_SESSION['captcha_admin']) {
    $error_msg = "Invalid Security Verification Code!";
    $_SESSION['captcha_admin'] = substr(str_shuffle("ABCDEFGHJKLMNPQRSTUVWXYZ23456789")
}
elseif (is_brute_force_blocked($email)) {
```

7) Session Fixation Protection



It generates a new session ID after a successful login to prevent session hijacking

attacks.

8) Dynamic capcha login

It generates a dynamic random code and verifies user input during form submission to prevent spamming and bot logins.

9) Csrf token

It shows a hidden input field containing a unique csrf_token value embedded inside the login form to secure form submissions.

Implementation:

1.8 User Interface (UI) Screenshots

(In this section, insert images of your system screens along with the following descriptions)

Login Page: Displays the dual-tab interface (Student vs. Admin/Staff), login fields, dynamic security captcha, and password toggle.

Password Reset Modal: Displays the pop-up form where students enter their email to receive a password reset link.

Student Dashboard: Displays cafeteria food items, prices, food categories, and order placement features.

Admin / Staff Dashboard: Displays order management tools, food menu updates, and system settings.

Login Page:

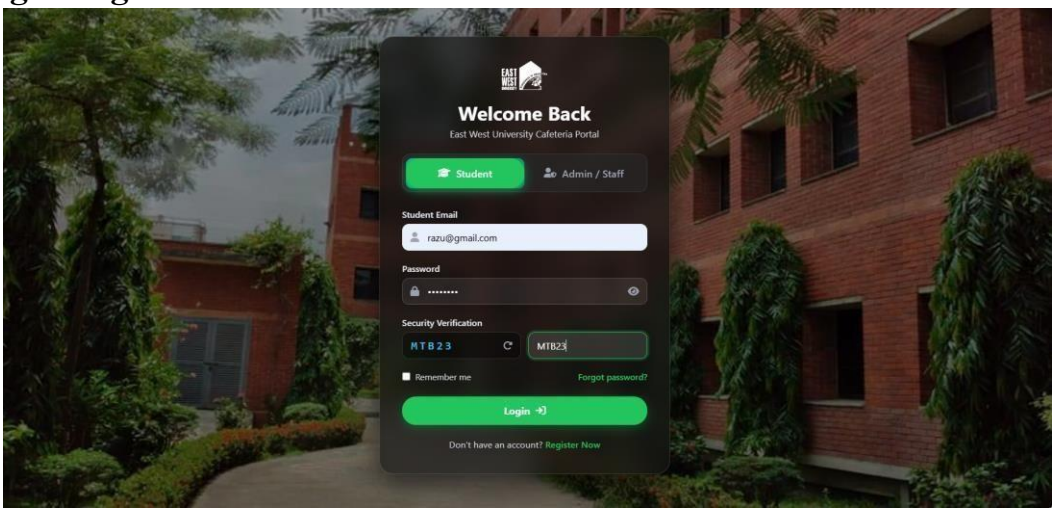


Figure: User Authentication & Login Interface

Password Reset Modal

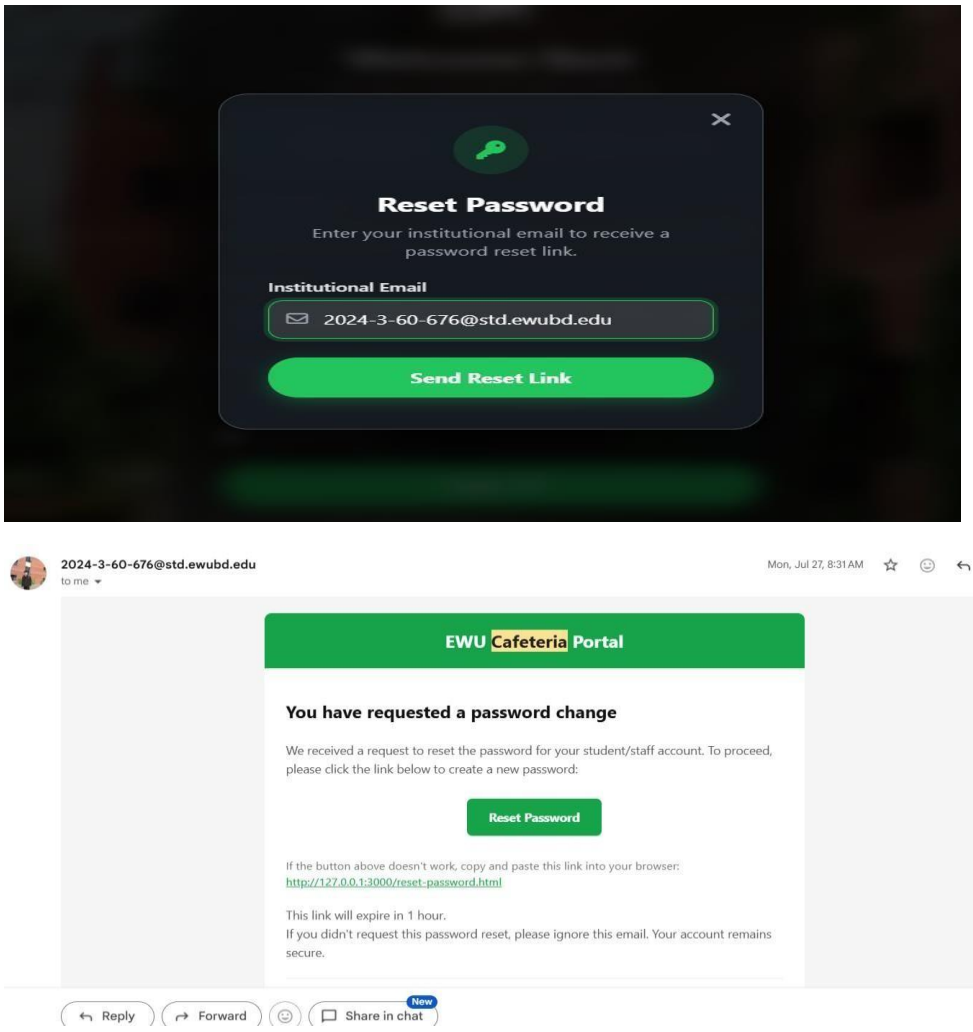


Figure: Password Recovery Modal Interface

Student Dashboard

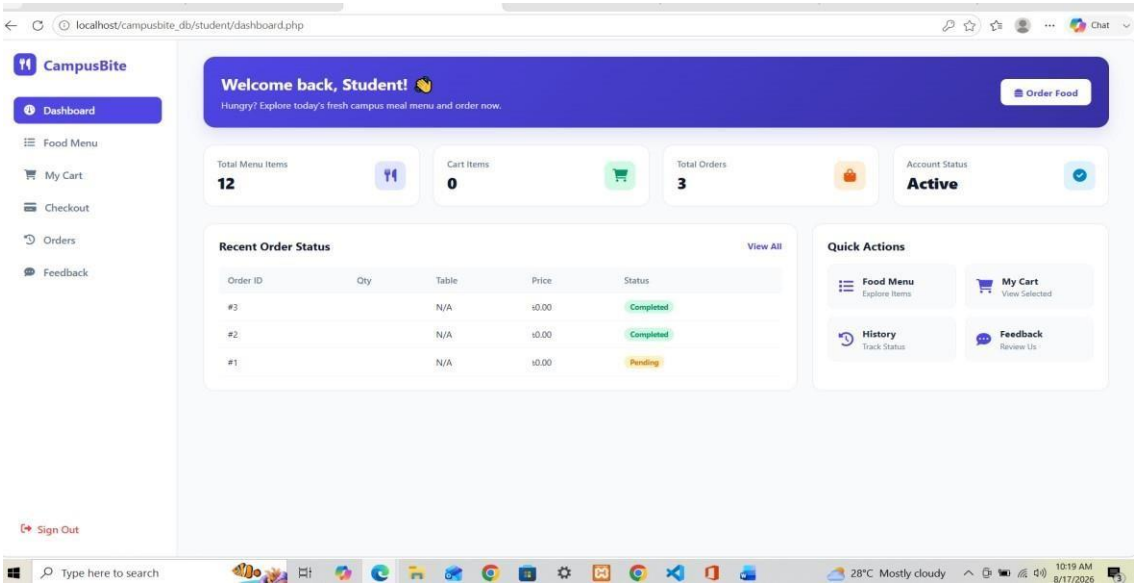


Figure: Student Dashboard Interface

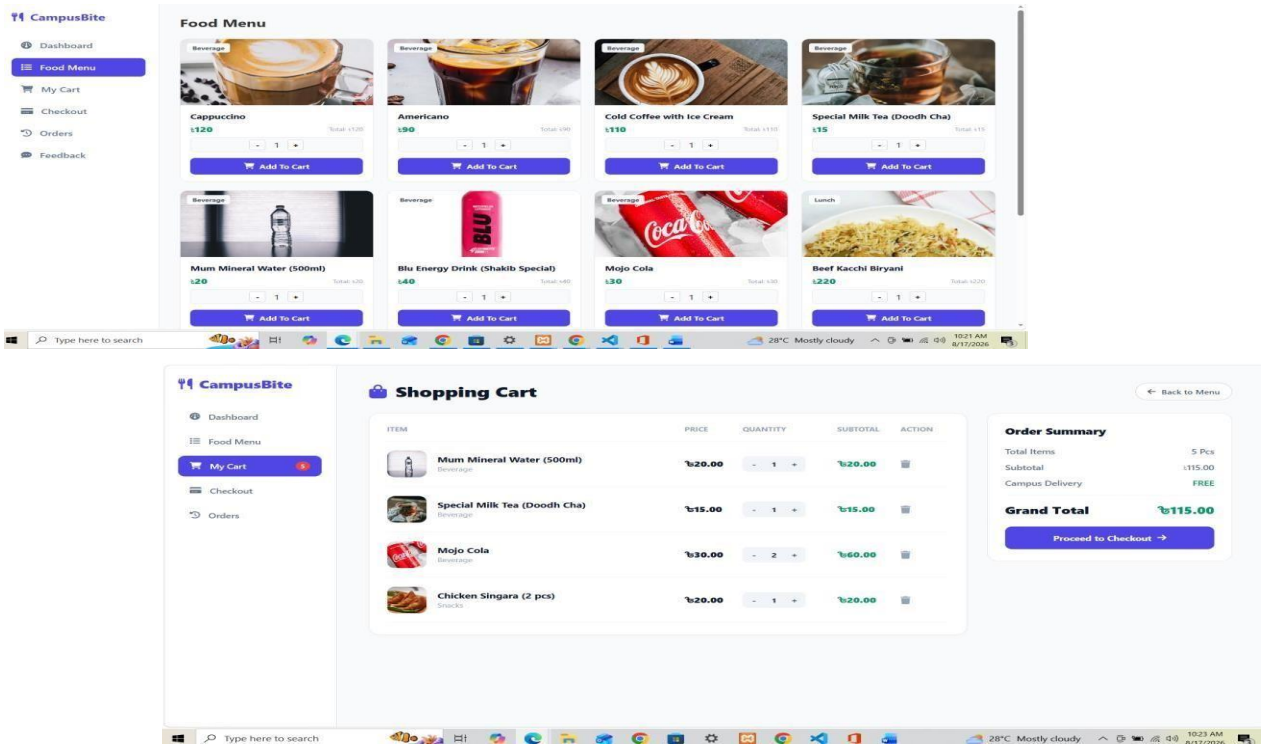


Figure 4: Food Menu Catalog Interface Shopping Cart & Order Summary View

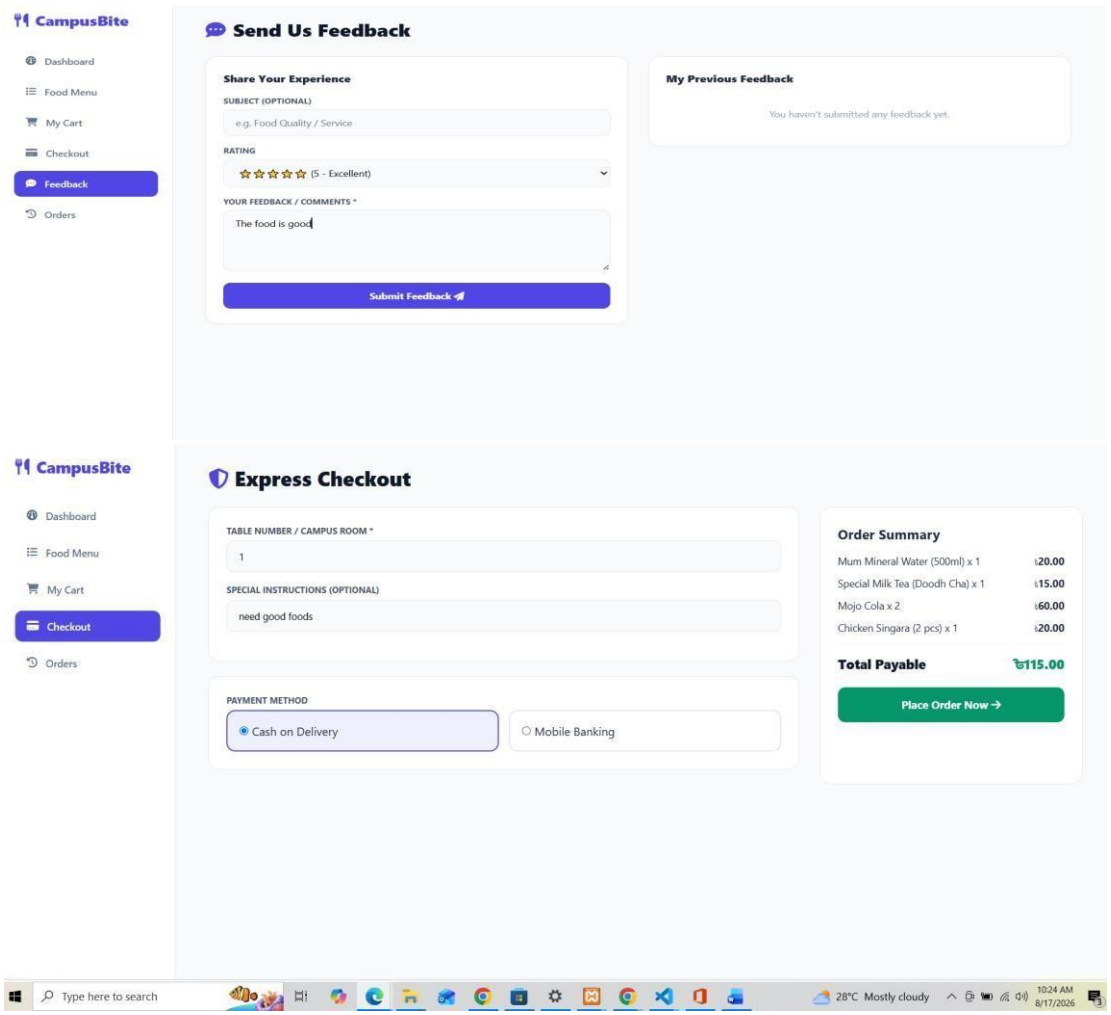


Figure Express Checkout & Payment View & Send Us Feedback Interface

Admin Login page

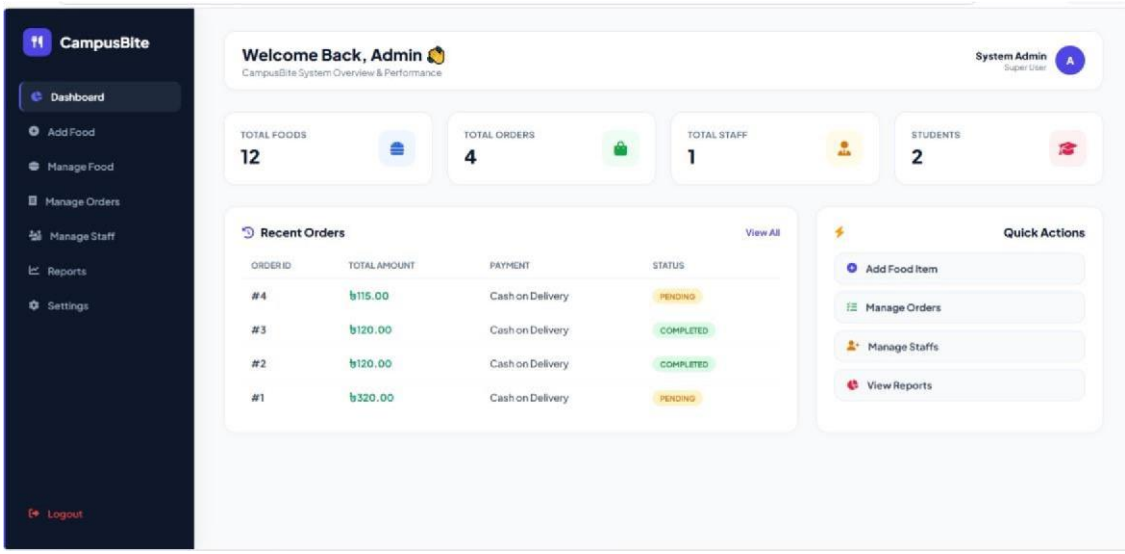


Figure :- Admin Dashboard Interface

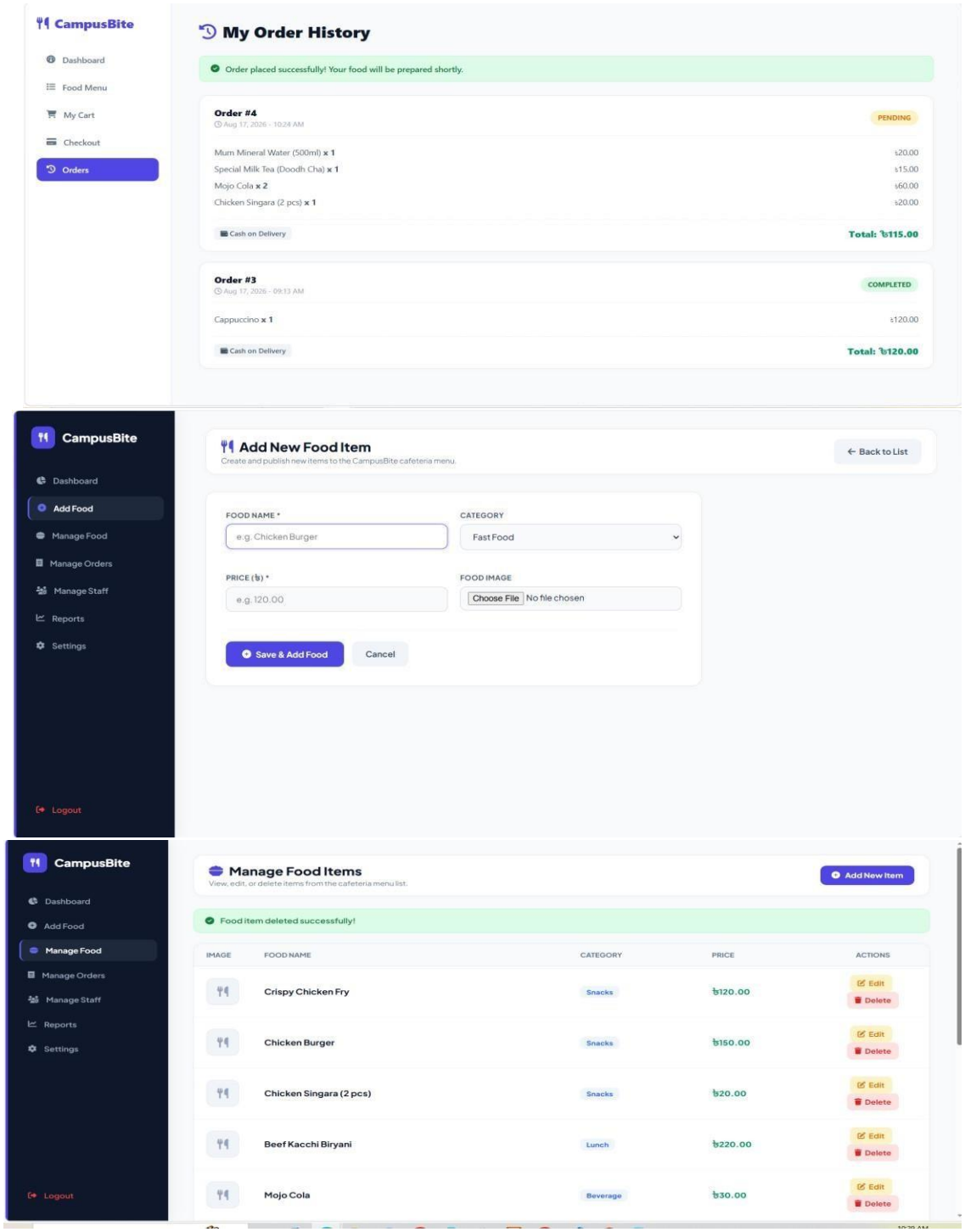


Figure:- My Order History Interface

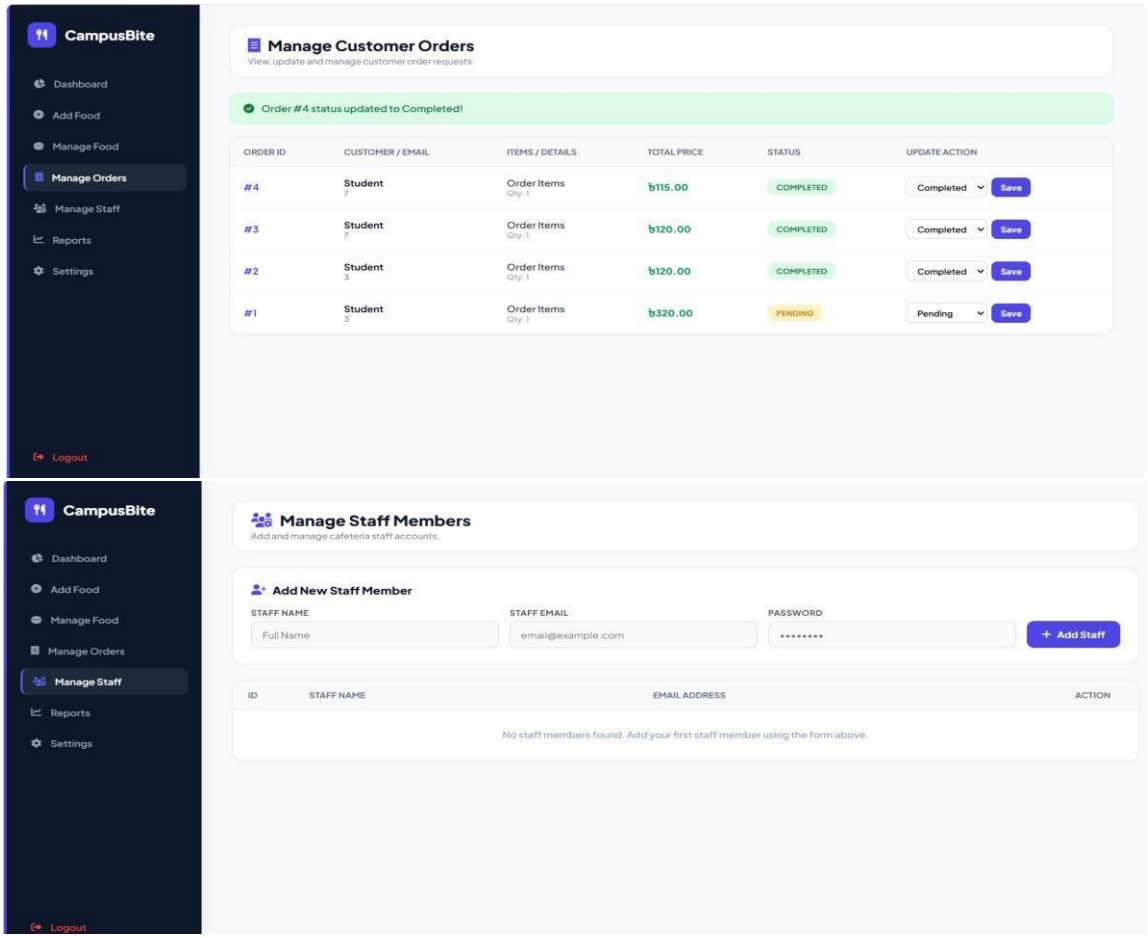


Figure 4.10: Manage Customer Orders and Staff Management Interfaces

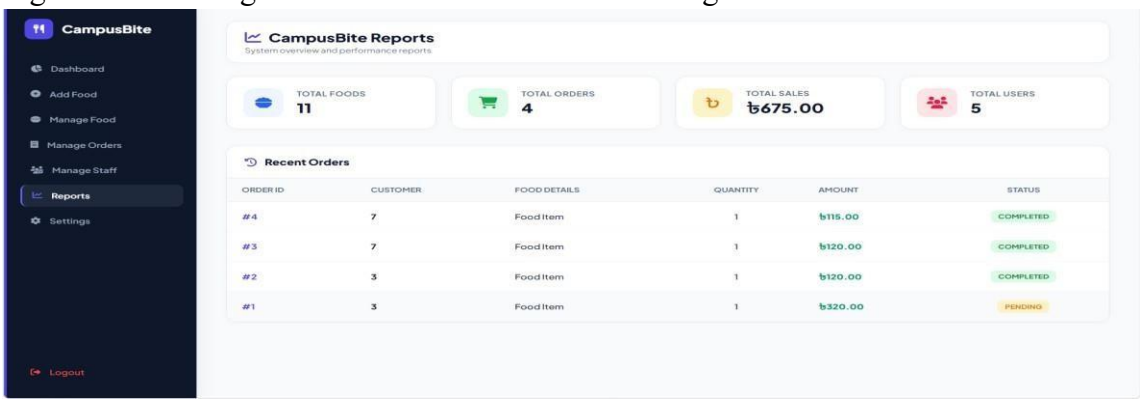


Figure: Administrative Reports and Analytics View

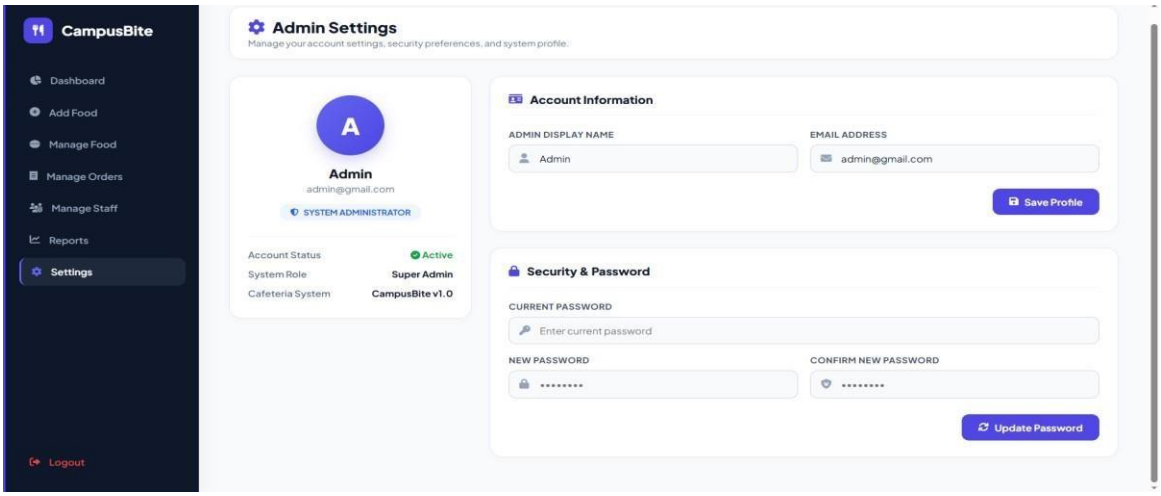


Figure:-Admin Settings and Account Security Interface

Conclusion

The East West University Cafeteria Portal has been developed with a security-first approach to ensure a resilient, reliable, and user-friendly digital environment. By systematically integrating multi-layered defensive controls—ranging from network-level HTTP security headers and prepared SQL statements to password hashing, session regeneration, brute-force protection, dynamic CAPTCHAs, and CSRF token validation—the system successfully mitigates primary web vulnerabilities like SQL Injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF). This robust security architecture guarantees data integrity, protects user confidentiality, and delivers a dependable administrative and purchasing experience for students, staff, and administrators alike.